

Augmented Physics-Informed Extreme Learning Machine to Solve the Biharmonic Equations via Fourier Expansions

Xi'an Li, Jinran Wu, Yujia Huang, Zhe Ding, Xin Tai, Liang Liu, You-Gan Wang

1 Introduction

Biharmonic equations represent an important class of fourth-order partial differential equations (PDEs) that frequently appear in elasticity theory, fluid mechanics, and many engineering applications. The general form of the biharmonic equation is:

$$\Delta^2 u(x) = f(x), \quad x \in \Omega \quad (1)$$

where Ω is a domain in $\mathbb{R}^d$ with a piecewise Lipschitz boundary, $f(x) \in L^2(\Omega)$ is a prescribed function, and Δ denotes the standard Laplace operator. The biharmonic operator $\Delta^2 u(x)$ is expressed as:

$$\Delta^2 u(x) = \sum_{i=1}^d \frac{\partial^4 u}{\partial x_i^4} + \sum_{i=1}^d \sum_{j=1, j \neq i}^d \frac{\partial^4 u}{\partial x_i^2 \partial x_j^2} \quad (2)$$

The solution is uniquely determined by one of the following boundary conditions:

- Dirichlet boundary conditions:

$$u(x) = g(x) \quad \text{and} \quad \frac{\partial u(x)}{\partial \vec{n}} = h(x) \quad \text{on} \quad \partial\Omega \quad (3)$$

- Navier boundary conditions:

$$u(x) = g(x) \quad \text{and} \quad \Delta u(x) = k(x) \quad \text{on} \quad \partial\Omega \quad (4)$$

Solving biharmonic equations is challenging due to the fourth-order derivative terms, particularly in complex geometries. Traditional numerical methods, such as finite element methods (FEM) and finite difference methods (FDM), often struggle with irregular domains and high-dimensional problems.

This paper introduces a Fourier-induced Physics-Informed Extreme Learning Machine (FPIELM) to address the limitations of conventional Physics-Informed Extreme Learning Machines (PIELM) when solving high-order PDEs such as the biharmonic equation. The FPIELM approach overcomes issues of parameter sensitivity and limited precision that affect PIELM implementations that use common activation functions (sigmoid, tangent, and Gaussian).

2 Background on Extreme Learning Machines

2.1 Extreme Learning Machine Framework

Extreme Learning Machine (ELM) is a type of single hidden layer fully connected neural network designed to establish a mapping $\ell : \mathbb{R}^d \rightarrow \mathbb{R}^k$. The distinguishing feature of ELM is that weights and biases in the input layer are randomly assigned and fixed throughout the training process, while only the output weights are optimized analytically.

Mathematically, the ELM mapping $\ell(x)$ with input $x \in \mathbb{R}^d$ and output $y \in \mathbb{R}^k$ is defined as:

$$y = (y_1, y_2, \dots, y_k) \quad \text{with} \quad y_m = \sum_{i=1}^n \beta_{mi} \sigma \left(\sum_{j=1}^d w_{ij} x_j + b_i \right) \quad (5)$$

where:

- $W = [W^{[1]}, W^{[2]}, \dots, W^{[n]}]^T \in \mathbb{R}^{n \times d}$ represents the weights of the hidden units
- $b = (b_1, b_2, \dots, b_n)^T \in \mathbb{R}^n$ is the bias vector
- $\sigma(\cdot)$ is the activation function applied element-wise
- $\beta = [\beta^{[1]}, \beta^{[2]}, \dots, \beta^{[k]}]^T \in \mathbb{R}^{k \times n}$ represents the weights connecting the hidden units to the outputs

2.2 Physics-Informed Extreme Learning Machine (PIELM)

PIELM incorporates physical laws into the ELM framework by optimizing a loss function that integrates governing equations and enforced boundary or initial conditions. Unlike Physics-Informed Neural Networks (PINNs), which rely on gradient descent-based optimization, PIELM analytically calculates the weights of the output layer, offering advantages in terms of accuracy, convergence, and computational efficiency.

ELM's universal approximation capability is a key advantage, as it has been proven that with N hidden nodes, an ELM can exactly learn N distinct samples, and with fewer than N hidden nodes, it can still achieve approximation within a permissible error.

3 Fourier-Induced PIELM for Biharmonic Equations

3.1 Vanilla PIELM for Biharmonic Equations

In the PIELM framework, the solution to the biharmonic equation is represented as:

$$u(x; W, b) = \sigma(x^T \cdot W^T + b^T) \cdot \beta \quad (6)$$

Substituting this into the biharmonic equation:

$$\Delta^2 \left(\sum_{i=1}^N \beta_i \sigma(V_i(x)) \right) = f(x) \quad \text{for } x \in \Omega \quad (7)$$

where $V_i(x) = x^T \cdot W^{[i]} + b_i$ is the linear output for the i -th hidden unit and N is the number of hidden units.

For Dirichlet boundary conditions, we have:

$$\sum_{i=1}^N \beta_i \sigma(V_i(x)) = g(x) \quad (8)$$

$$\frac{\partial}{\partial \vec{n}} \left(\sum_{i=1}^N \beta_i \sigma(V_i(x)) \right) = h(x) \quad \text{for } x \in \partial\Omega \quad (9)$$

For Navier boundary conditions, the second equation is replaced by:

$$\Delta \left(\sum_{i=1}^N \beta_i \sigma(V_i(x)) \right) = k(x) \quad \text{for } x \in \partial\Omega \quad (10)$$

The PIELM approach requires sampling collocation points from both the interior and boundaries of the domain. Let Q denote the number of collocation points in the interior $S_I = \{x_I^q\}_{q=1}^Q$, and P denote the number of collocation points on the boundary $S_B = \{x_B^p\}_{p=1}^P$.

For Dirichlet boundaries, the residual terms are:

$$R_u(x_I^q) = \Delta^2 \left(\sum_{i=1}^N \beta_i \sigma(V_i(x_I^q)) \right) - f(x_I^q) \quad (11)$$

$$L_g(x_B^p) = \sum_{i=1}^N \beta_i \sigma(V_i(x_B^p)) - g(x_B^p) \quad (12)$$

$$L_h(x_B^p) = \frac{\partial}{\partial \vec{n}} \left(\sum_{i=1}^N \beta_i \sigma(V_i(x_B^p)) \right) - h(x_B^p) \quad (13)$$

For Navier boundaries, the last term is replaced by:

$$L_k(x_B^p) = \Delta \left(\sum_{i=1}^N \beta_i \sigma(V_i(x_B^p)) \right) - k(x_B^p) \quad (14)$$

To approximate the solution well, the output weights of the PIELM model need to be adjusted to minimize the error of the following system of linear equations:

$$H\beta = S \Leftrightarrow \begin{bmatrix} H_f \\ H_g \\ H_h \end{bmatrix} \beta = \begin{bmatrix} S_f \\ S_g \\ S_h \end{bmatrix} \quad \text{or} \quad \begin{bmatrix} H_f \\ H_g \\ H_k \end{bmatrix} \beta = \begin{bmatrix} S_f \\ S_g \\ S_k \end{bmatrix} \quad (15)$$

The matrices H_f , H_g , H_h , and H_k include terms related to the activation function and its derivatives, evaluated at the collocation points. The vectors S_f , S_g , S_h , and S_k contain the values of f , g , h , and k at the respective collocation points.

When the weights and biases of the hidden layer are randomly initialized and fixed, the output weights β are obtained by solving the linear system $H\beta = S$, which can be done in several ways:

- If H is square and invertible: $\beta = H^{-1}S$
- If H is rectangular and full rank: $\beta = H^\dagger S$, where $H^\dagger = (H^T H)^{-1} H^T$ is the pseudo-inverse
- If H is singular: $\beta = (H^T H + \lambda I)^{-1} H^T S$, where $\lambda > 0$ is the Tikhonov regularization parameter

3.2 Choice of Activation Function for Biharmonic Equations

The choice of activation function is critical for the performance of the ELM model, especially for high-order PDEs like the biharmonic equation. The paper identifies a key issue with common activation functions (sigmoid, Gaussian, and tangent) when dealing with high-order derivatives:

1. These functions and their derivatives are bounded and tend to saturate for narrow-range inputs.
2. Their performance degrades with large-range inputs, as the fourth-order derivatives of these functions exhibit complex behavior.

The sigmoid, Gaussian, and tangent functions satisfy Lipschitz continuity and exhibit bounded outputs, but their high-order derivatives can be problematic.

To address this limitation, the paper proposes using Fourier-type functions, specifically the sine function, as an alternative activation function. The sine function serves as an ideal global Fourier basis function with the following advantages:

1. It can generate responses along with its derivatives for both narrow and large-range inputs.
2. Approximations based on such basis functions can handle data across various scales.
3. The derivatives of sine functions have a consistent pattern, making them computationally simpler for high-order problems.

3.3 FPIELM Algorithm

The complete FPIELM algorithm for solving the biharmonic equation is outlined as follows:

Algorithm 1 FPIELM algorithm for solving biharmonic equation

- 1: Generate the points set S including interior points $S_I = \{x_I^q\}_{q=1}^Q$ and boundary points $S_B = \{x_B^p\}_{p=1}^P$
 - 2: Select a shallow neural network with sine activation function and randomly initialize weights and biases of the hidden layer from a uniform distribution on $[-\delta, \delta]$
 - 3: Fix the parameters of the hidden layer
 - 4: Embed the input data into the implicit feature space by a linear transformation, activate nonlinearly the results and cast them into the output layer
 - 5: Combine linearly all output features of the hidden layer and output this result
 - 6: Compute the derivatives of the output with respect to the input variables and construct the linear system
 - 7: Learn the output layer weights by the least-squares method
-

The key innovation of FPIELM is the use of the sine function as the activation function, which allows for better handling of high-order derivatives in the biharmonic equation.

4 Numerical Examples and Results

The paper presents extensive numerical experiments to demonstrate the effectiveness of the FPIELM method compared to PIELM implementations using sigmoid (SPIELM), Gaussian (GPIELM), and hyperbolic tangent (TPIELM) activation functions.

4.1 Experimental Setup

The performance evaluation uses the relative error (REL) criterion:

$$\text{REL} = \sqrt{\frac{\sum_{i=1}^{N'} |\tilde{u}(x_i) - u^*(x_i)|^2}{\sum_{i=1}^{N'} |u^*(x_i)|^2}} \quad (16)$$

where $\tilde{u}(x_i)$ is the approximate solution and $u^*(x_i)$ is the exact solution at the collection points $\{x_i\}_{i=1}^{N'}$.

All experiments were implemented in NumPy (version 1.21.0) on a workstation with 256 GB RAM and an NVIDIA GeForce RTX 4090 Ti GPU.

4.2 Dirichlet Boundary Condition Examples

4.2.1 Example 1: Regular Domains

The first example examines the biharmonic equation with an exact solution given by:

$$u(x_1, x_2) = [(x_1 - x_1^{\min})(x_1^{\max} - x_1)]^2 [(x_2 - x_2^{\min})(x_2^{\max} - x_2)]^2 \quad (17)$$

in four different regular domains: $[-1, 1] \times [-1, 1]$, $[0, 5] \times [0, 5]$, $[-4, 6] \times [-3, 7]$, and $[5, 15] \times [0, 10]$.

Key results include:

- FPIELM consistently achieves lower relative errors than SPIELM, GPIELM, and TPIELM across all domains.
- On the domain $[-1, 1] \times [-1, 1]$, FPIELM achieves $\text{REL} = 2.223 \times 10^{-13}$, outperforming the other methods.
- FPIELM is less sensitive to the scale factor δ for initializing weights and biases.
- FPIELM's performance stabilizes as the number of hidden neurons increases.
- FPIELM is more computationally efficient, with shorter runtime.

The paper also compares FPIELM with the finite difference method (FDM) on the domain $[0, 5] \times [0, 5]$, showing that FPIELM maintains stability across various mesh grid sizes and significantly outperforms FDM in terms of accuracy, particularly on sparse grids.

4.2.2 Example 2: Hexagram Domains

The second example tests the biharmonic equation with Dirichlet boundary conditions in hexagram-shaped domains derived from $[-\pi, \pi] \times [-\pi, \pi]$ and $[0, 3\pi] \times [\pi, 2\pi]$. The exact solution is:

$$u(x_1, x_2) = \sin(x_1)e^{\cos(x_2)} \quad (18)$$

The results show:

- FPIELM achieves the best accuracy on both hexagram domains with $\text{REL} = 3.876 \times 10^{-10}$ on the domain derived from $[-\pi, \pi] \times [-\pi, \pi]$.
- FPIELM shows more stable performance as the number of hidden nodes and the scale factor δ vary.
- FPIELM is more robust on non-unitized domains where other methods struggle.

4.2.3 Example 3: Three-Dimensional Domain with Holes

The third example involves a three-dimensional cubic domain $[1, 3] \times [1, 3] \times [1, 3]$ containing multiple holes. The exact solution is:

$$u(x_1, x_2, x_3) = 50e^{-0.25(x_1+x_2+x_3)} \quad (19)$$

The results demonstrate:

- FPIELM outperforms the other three methods in this complex three-dimensional domain.
- FPIELM remains stable across various scale factors δ and achieves higher accuracy.
- The coefficient matrix in the final linear system is full rank, eliminating the need for Tikhonov regularization.

4.3 Navier Boundary Condition Examples

4.3.1 Example 4: Regular Rectangular Domains

The fourth example deals with the biharmonic equation with Navier boundary conditions in rectangular domains $[0, 1] \times [0, 1]$ and $[0, 4] \times [0, 4]$. The exact solution is:

$$u(x_1, x_2) = \sin(x_1^2 + x_2^2) \quad (20)$$

The results show:

- FPIELM achieves extremely high accuracy with $\text{REL} = 1.891 \times 10^{-15}$ on the domain $[0, 1] \times [0, 1]$.

- On the non-unitized domain $[0, 4] \times [0, 4]$, FPIELM ($\text{REL} = 9.418 \times 10^{-8}$) maintains good accuracy while the other methods struggle ($\text{REL} \geq 0.09$).
- FPIELM is less sensitive to the scale factor δ and number of hidden nodes.

4.3.2 Example 5: Porous Domains

The fifth example considers the biharmonic equation on porous domains derived from $[-1, 1] \times [-\pi, \pi]$ and $[0, 4] \times [0, 4\pi]$. The exact solution is:

$$u(x_1, x_2) = e^{x_1} \sin(x_2) \quad (21)$$

The findings include:

- FPIELM achieves $\text{REL} = 2.072 \times 10^{-9}$ on the domain derived from $[-1, 1] \times [-\pi, \pi]$, outperforming other methods.
- On the non-unitized domain, FPIELM ($\text{REL} = 5.581 \times 10^{-4}$) significantly outperforms other methods that fail to converge ($\text{REL} \geq 0.6$).
- FPIELM shows stability across a broad range of scale factors δ .

4.3.3 Example 6: Three-Dimensional Spherical Domain

The final example addresses the biharmonic equation in a three-dimensional space bounded by two spherical surfaces with radii $r_1 = 0.2$ and $r_2 = 1.0$. The exact solution is:

$$u(x_1, x_2, x_3) = \sin(\pi x_1) \sin(\pi x_2) \sin(\pi x_3) \quad (22)$$

The results demonstrate:

- FPIELM produces lower errors than the other methods in this complex three-dimensional domain.
- FPIELM remains stable and robust across varying scale factors δ , with a notably broad effective range.
- The runtime of FPIELM increases linearly as the number of hidden nodes gradually grows.

5 Implications and Advantages

The paper’s findings have several important implications:

1. **Improved Accuracy for High-Order PDEs:** FPIELM significantly outperforms PIELM implementations with traditional activation functions (sigmoid, Gaussian, and tangent) when solving the bi-harmonic equation, achieving orders of magnitude higher accuracy.
2. **Enhanced Stability and Robustness:** FPIELM demonstrates greater stability across a wider range of scale factors δ for weight initialization and is less sensitive to the number of hidden neurons.
3. **Better Performance on Non-Unitized Domains:** While traditional PIELM methods struggle or fail on non-unitized domains, FPIELM maintains high accuracy, making it more versatile for practical applications.
4. **Computational Efficiency:** FPIELM requires less computation time compared to other PIELM implementations and can solve problems that traditional numerical methods like FDM struggle with.
5. **Effectiveness in Complex Geometries:** The method successfully handles complex geometries, including irregular domains, domains with holes, and three-dimensional problems, without requiring mesh generation.
6. **Flexibility:** FPIELM works effectively with both Dirichlet and Navier boundary conditions, showing its flexibility in addressing different types of boundary value problems.

The superiority of the Fourier-based activation function for high-order PDEs suggests a promising direction for other PDE applications, especially those involving high-order derivatives and complex geometries. This approach overcomes limitations of traditional neural network methods while maintaining the advantages of the ELM framework, such as mesh-free operation, rapid learning, and parallel architecture compatibility.